



PHASE 2 | TASK 4

Airflow Runs the Scheduled Work

The stream never stops; an hourly DAG guards its freshness, compacts its files and rolls up the day



STEP 1

Start Airflow in Docker

WHAT

Ran the official Airflow 3.3.2 docker-compose stack on Docker Desktop with the WSL 2 backend. The Databricks provider installs at start-up, and a Fernet key in a git-ignored .env encrypts stored connections.

WHY

Airflow is seven cooperating services, not one program: API server and UI, scheduler, DAG processor, worker, triggerer, Postgres for run history and Redis as the task queue. Docker runs them all on one laptop exactly as they would run on a server.

STEP 1 | Start Airflow in Docker

```
Windows PowerShell
/opt/airflow/logs:
total 4
drwxrwxrwx 1 airflow root 4096 Oct  1 08:45 .
drwxrwxr-x 1 airflow root 4096 Oct  1 21:30 ..

/opt/airflow/plugins:
total 4
drwxrwxrwx 1 airflow root 4096 Oct  1 08:45 .
drwxrwxr-x 1 airflow root 4096 Oct  1 21:30 ..

PS F:\Github Cloned Repos\atliq-commerce-capstone\phase2-streaming\airflow> docker compose up -d
[+] 8/8/8
Container airflow-redis-1 Healthy
Container airflow-airflow-dag-processor-1 Started
Container airflow-postgres-1 Healthy
Container airflow-airflow-apiserver-1 Healthy
Container airflow-airflow-triggerer-1 Started
Container airflow-airflow-scheduler-1 Started
Container airflow-airflow-worker-1 Started
Container airflow-airflow-init-1 Exited

PS F:\Github Cloned Repos\atliq-commerce-capstone\phase2-streaming\airflow> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE                                CREATED                                STATUS                                PORTS
airflow-airflow-apiserver-1         apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-apiserver                    About a minute ago                  Up About a minute (healthy)         0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
airflow-airflow-dag-processor-1     apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-dag-processor                 About a minute ago                  Up About a minute (healthy)         8080/tcp
airflow-airflow-scheduler-1         apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-scheduler                    About a minute ago                  Up About a minute (health: starting) 8080/tcp
airflow-airflow-triggerer-1         apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-triggerer                    About a minute ago                  Up About a minute (healthy)         8080/tcp
airflow-airflow-worker-1           apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-worker                        About a minute ago                  Up 5 seconds (health: starting)      8080/tcp
airflow-postgres-1                 postgres:16                         "docker-entrypoint.s..."          postgres                             5 minutes ago                      Up 5 minutes (healthy)              5432/tcp
airflow-redis-1                     redis:7.2-bookworm                  "docker-entrypoint.s..."          redis                                5 minutes ago                      Up 5 minutes (healthy)              6379/tcp

PS F:\Github Cloned Repos\atliq-commerce-capstone\phase2-streaming\airflow> docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE                                CREATED                                STATUS                                PORTS
airflow-airflow-apiserver-1         apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-apiserver                    3 minutes ago                      Up 3 minutes (healthy)              0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
airflow-airflow-dag-processor-1     apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-dag-processor                 3 minutes ago                      Up 3 minutes (healthy)              8080/tcp
airflow-airflow-scheduler-1         apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-scheduler                    3 minutes ago                      Up 3 minutes (healthy)              8080/tcp
airflow-airflow-triggerer-1         apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-triggerer                    3 minutes ago                      Up 3 minutes (healthy)              8080/tcp
airflow-airflow-worker-1           apache/airflow:3.3.2               "/usr/bin/dumb-init ..."          airflow-worker                        3 minutes ago                      Up 2 minutes (healthy)              8080/tcp
airflow-postgres-1                 postgres:16                         "docker-entrypoint.s..."          postgres                             7 minutes ago                      Up 7 minutes (healthy)              5432/tcp
airflow-redis-1                     redis:7.2-bookworm                  "docker-entrypoint.s..."          redis                                7 minutes ago                      Up 7 minutes (healthy)              6379/tcp

PS F:\Github Cloned Repos\atliq-commerce-capstone\phase2-streaming\airflow>
```

docker compose ps: all seven services healthy.



STEP 2

Connect Airflow to Databricks

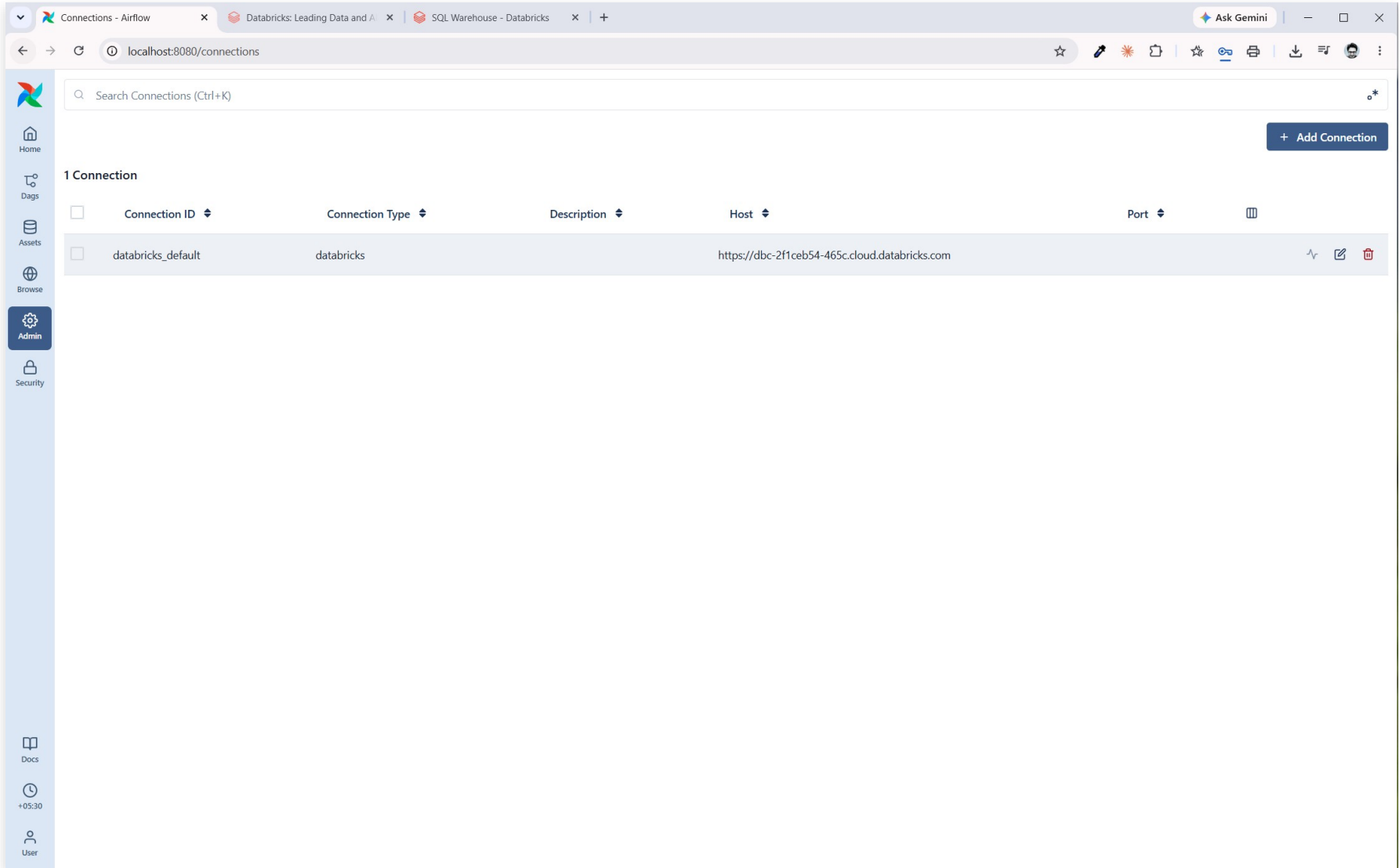
WHAT

Created the connection databricks_default with the Free Edition workspace host and a personal access token scoped to SQL only, valid for 30 days.

WHY

The token lives encrypted in Airflow's metadata database, never in the DAG file. Scoping it to SQL means Airflow can run warehouse queries and nothing else: the same least-privilege rule as Phase 1.

STEP 2 | Connect Airflow to Databricks



Connection databricks_default, type databricks; the token is never displayed.



STEP 3

Build the Streaming Ops DAG

WHAT

Completed the hourly DAG `atliq_streaming_ops`: three `DatabricksSqlOperator` tasks chained `check_fresh_events`, then `optimize_tables`, then `refresh_daily_summary`, all running on the Serverless Starter Warehouse.

WHY

The gate runs first so maintenance and the rollup never run on stale data. The gate has `retries=0` because stale data will not fix itself in five minutes; the other tasks keep two retries for transient failures.

STEP 3 | Build the Streaming Ops DAG (1 of 2)

Task 4: streaming ops DAG (DQ gate, OPTIMIZE, daily rollup) and Airflow compose

main

2 files changed

Filter files...

phase2-streaming/airflow

dags

atliq_streaming_ops_dag.py

docker-compose.yaml

Search within code

phase2-streaming/airflow/dags/atliq_streaming_ops_dag.py

@@ -1,25 +1,20 @@

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

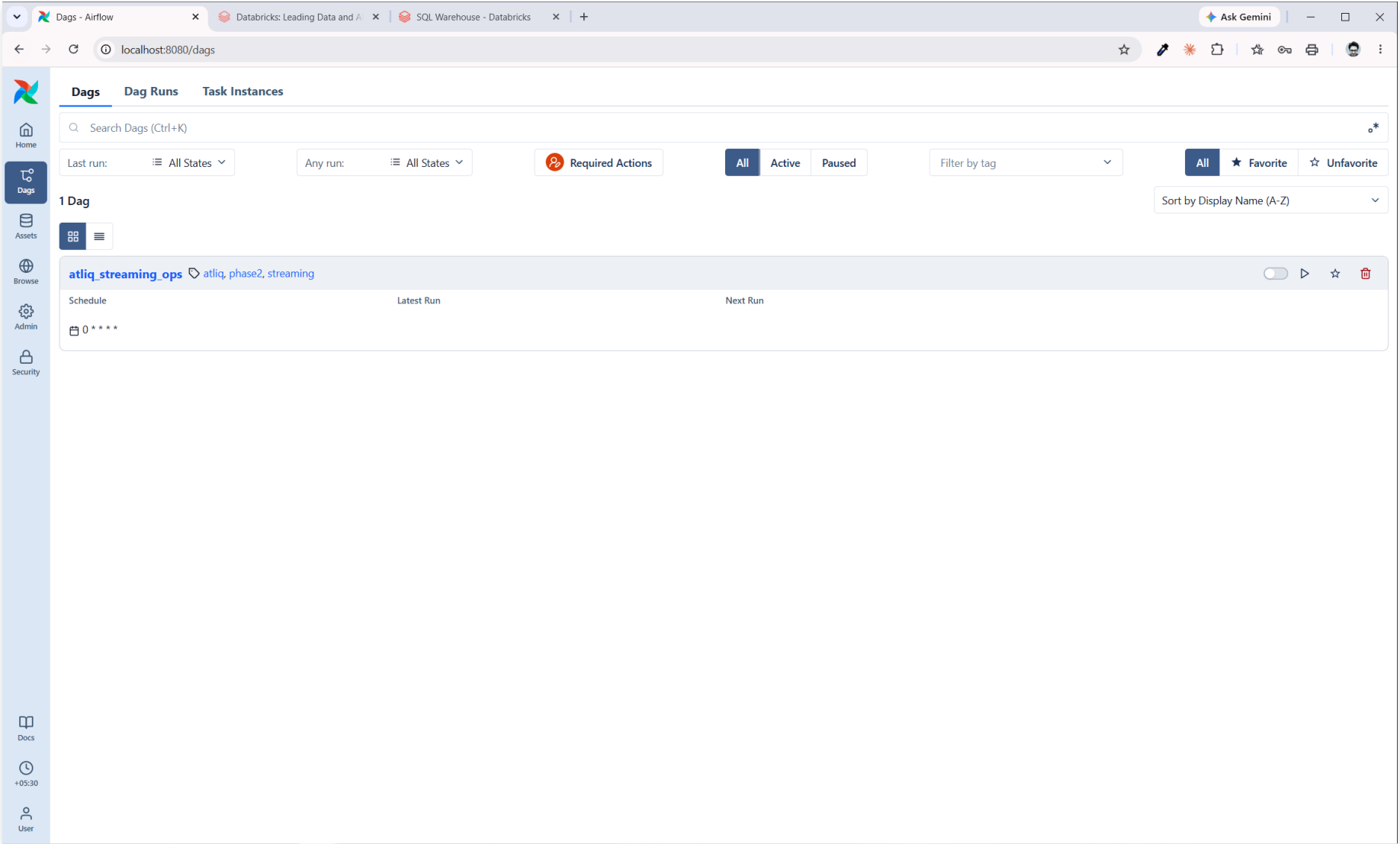
34

35

36

Commit b73e4af: the starter replaced by the three operators, Airflow 3 import and warehouse path.

STEP 3 | Build the Streaming Ops DAG (2 of 2)



atliq_streaming_ops parsed with no import errors: hourly schedule (0 * * * *), tags atliq, phase2, streaming.



STEP 4

Green Run on Fresh Data

WHAT

Checked Silver's freshness first: the newest event was 99 minutes old, inside the 2-hour limit. Triggered the DAG; all three tasks succeeded in 24.8 seconds.

WHY

Measuring freshness before the run makes the outcome predictable rather than lucky. Under 120 minutes the gate must pass; over 120 it must fail. The next steps prove both sides.

STEP 4 | Green Run on Fresh Data (1 of 2)

databricks

Free Edition

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Home

Tree view: ON

ashish@alp...

Type to search

01_kafka_stream_processing

Bakehouse Sales Starter Space

01_kafka_stream_processing

File Edit View Run Help Python Full Last edit was now

1	2026-10-01T20:20:00.000+00:...	269	745748.00	745748.00	0.00
2	2026-10-01T20:15:00.000+00:...	23	82653.00	82653.00	0.00
3	2026-10-01T19:15:00.000+00:...	104	236259.00	236259.00	0.00
4	2026-10-01T07:55:00.000+00:...	25	76654.00	76654.00	0.00
5	2026-10-01T07:50:00.000+00:...	256	684038.00	684038.00	0.00

5 rows2.804s runtime

Refreshed at 10/2/26, 2:20 AM (1h ago)

This result is stored as `_sql1df` and can be used in other Python and SQL cells.

Just now (16s)

22

SQL

```
%sql
SELECT MAX(event_ts) AS last_event, current_timestamp() AS now,
       timestampdiff(MINUTE, MAX(event_ts), current_timestamp()) AS minutes_old
FROM atliq.streaming.silver_order_events;
```

See performance (1)

Checked for improvements

Table

	last_event	now	minutes_old
1	2026-10-01T20:39:34.502+00:...	2026-10-01T22:19:18.932+00:...	99

1 row16.256s runtime

Refreshed at 10/2/26, 3:49 AM (Now)

This result is stored as `_sql1df` and can be used in other Python and SQL cells.

Before the green run: last event 20:39 UTC, minutes_old 99.

STEP 4 | Green Run on Fresh Data (2 of 2)

Home

Dags

Assets

Browse

Admin

Security

Docs

+05:30

User

manual__2026-10-01T22:21:16...

01_kafka_stream_processing -

+

Ask Gemini

localhost:8080/dags/atliq_streaming_ops/runs/manual__2026-10-01T22:21:16.651696+00:00

Search Dags Ctrl+K

Trigger

Dag

atliq_streaming_ops

Dag Run

manual__2026-10-01T22:21:16.651696+00:00

check_fresh_events

DatabricksSqlOperator

success

optimize_tables

DatabricksSqlOperator

success

refresh_daily_summary

DatabricksSqlOperator

success

manual__2026-10-01T22:21:16.651696+00:00

Success

Logical Date

2026-10-02 03:50:36

Run Type

Manual

Start Date

2026-10-02 03:51:16

End Date

2026-10-02 03:51:41

Duration

00:00:24.793

Triggering User

airflow

Dag Version(s)

v1

Add a note...

+ Filter

3 Task Instances

Task ID

Map Index

State

refresh_daily_summary

Success

optimize_tables

Success

check_fresh_events

Success

Manual run: check_fresh_events, optimize_tables, refresh_daily_summary all success in 24.8 s.



STEP 5

What Airflow Did in Databricks

WHAT

Databricks Query History records every statement the DAG sent, with source Databricks Airflow and Airflow query tags. The rebuilt gold_daily_summary carries refreshed_at 22:21:39 UTC, inside the run's window.

WHY

A green box in Airflow says the task finished; the warehouse's own log says what actually ran. Daily counts of 1,748 placed, 1,445 paid and 323 cancelled match Silver exactly, so the rollup reconciles with its source.

STEP 5 | What Airflow Did in Databricks (1 of 2)

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

Query History

User: Me (ashish@alphainterbiz.com) Last 7 days Compute Duration Status Statement Source Statement ID 58 queries Reset filters

Query	Started at	Duration	Source
> SELECT version, timestamp, operation, operationMetrics.numRemovedFile...	10/02/2026, 03:59:09 AM	3.38 s	optimize_history
> CREATE OR REPLACE TABLE atliq.streaming.gold_daily_summary AS SELECT ...	10/02/2026, 03:56:46 AM	2.40 s	Databricks Airflow
> SELECT * FROM atliq.streaming.gold_daily_summary ORDER BY event_date ...	10/02/2026, 03:55:32 AM	1.07 s	daily_summary
> CREATE OR REPLACE TABLE atliq.streaming.gold_daily_summary AS SELECT ...	10/02/2026, 03:51:37 AM	4.22 s	Databricks Airflow
> CREATE OR REPLACE TABLE atliq.streaming.gold_daily_summary AS SELECT ...	10/02/2026, 03:51:36 AM	5.23 s	Databricks Airflow
> OPTIMIZE atliq.streaming.gold_revenue_5min	10/02/2026, 03:51:32 AM	1.53 s	Databricks Airflow
> OPTIMIZE atliq.streaming.silver_order_events	10/02/2026, 03:51:28 AM	2.61 s	Databricks Airflow
> OPTIMIZE atliq.streaming.gold_revenue_5min	10/02/2026, 03:51:26 AM	5.17 s	Databricks Airflow
Expand query text			
> SELECT assert_true(COUNT(*) > 0, 'DQ FAIL: no events in atliq.stream...	10/02/2026, 03:51:22 AM	874 ms	Databricks Airflow
> OPTIMIZE atliq.streaming.silver_order_events	10/02/2026, 03:51:17 AM	6.81 s	Databricks Airflow
> SELECT assert_true(COUNT(*) > 0, 'DQ FAIL: no events in atliq.stream...	10/02/2026, 03:50:49 AM	19.72 s	Databricks Airflow
> SELECT MAX(event_ts) AS last_event, current_timestamp() AS now, times...	10/02/2026, 03:49:18 AM	7.47 s	> 01_kafka_stream_processing / Cell(ID: 6987356650975696)
> spark.sql(f"CREATE VOLUME IF NOT EXISTS {CATALOG}.{SCHEMA}.checkpoint...	10/02/2026, 03:48:47 AM	793 ms	> 01_kafka_stream_processing / Cell(ID: 79073...980818606): 18
> spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{SCHEMA}")	10/02/2026, 03:48:45 AM	2.19 s	> 01_kafka_stream_processing / Cell(ID: 79073...980818606): 17
> spark.sql(f"CREATE CATALOG IF NOT EXISTS {CATALOG}")	10/02/2026, 03:48:42 AM	3.16 s	> 01_kafka_stream_processing / Cell(ID: 79073...980818606): 16
> SELECT g.window_start, g.orders_paid, g.revenue AS gold_revenue, s.re...	10/02/2026, 02:20:20 AM	1.29 s	> 01_kafka_stream_processing / Cell(ID: 8953089384897901)
> SELECT * FROM atliq.streaming.gold_revenue_5min ORDER BY window_start...	10/02/2026, 02:18:01 AM	491 ms	> 01_kafka_stream_processing / Cell(ID: 7907393980818615)
> SELECT event_type, COUNT(*) AS events FROM atliq.streaming.silver_ord...	10/02/2026, 02:17:47 AM	790 ms	> 01_kafka_stream_processing / Cell(ID: 7907393980818614)
> SELECT MAX(window_end) AS latest_closed_window, current_timestamp() A...	10/02/2026, 02:16:26 AM	622 ms	> 01_kafka_stream_processing / Cell(ID: 8953089384897898)
> print("Gold windows:", spark.table(GOLD).count())	10/02/2026, 02:14:58 AM	360 ms	> 01_kafka_stream_processing / Cell(ID: 79073...980818610): 25
> gold_q = (gold_df.writeStream .outputMode("append") # a window is wr...	10/02/2026, 02:14:32 AM	25.84 s	> 01_kafka_stream_processing / Cell(ID: 79073...980818610): 22
> print("Silver rows:", spark.table(SILVER).count())	10/02/2026, 02:14:12 AM	831 ms	> 01_kafka_stream_processing / Cell(ID: 79073...980818612): 33
> silver_q = (silver_df.writeStream .outputMode("append") .option("che...	10/02/2026, 02:13:29 AM	42.73 s	> 01_kafka_stream_processing / Cell(ID: 79073...980818612): 30

Finished

ashish@alphainterbiz.com · Serverless Starter Warehouse · Run on DBSQL

ID: 01f1bde6-7122-1e13-918d-ce95efadb643

OPTIMIZE atliq.streaming.gold_revenue_5min

See query text

Query wall-clock duration

Total wall-clock duration 5 s 169 ms

Start time Oct 02, 2026, 03:51:26 AM GMT+05:30

End time Oct 02, 2026, 03:51:31 AM GMT+05:30

Result fetching by client 1.35 s

Bytes read 7.67 KB 0%

Bytes written 2.59 KB

Rows read 5

Rows written 5

Files read 5 0%

Files written 1

See query profile

See longest operations for this query

Query Source

Databricks Airflow

Query tags

airflow,... : atliq_stream... airflow_map_index: -1 +3

Aggregated task time

Tasks total time 1.14 s

Tasks time in Photon 63 %

IO

Rows returned 1

Rows read 5

Bytes read 7.67 KB

Bytes pruned 0 bytes

Bytes read from cache 0 %

Bytes written 2.59 KB

Rows written 5

Files & partitions

See query profile

Query History: the gate, two OPTIMIZEs and the rollout, source Databricks Airflow; OPTIMIZE on Gold read 5 files, wrote 1.

STEP 5 | What Airflow Did in Databricks (2 of 2)

databricks

Free Edition

+ New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

SQL Editor

Tree view: ON

Workspace

Type to search

Repos

Shared

Users

daily_summary

Run all (1000)

Just now (1s)

workspace.default

Last edit was now

Generate

Serverless Starte...

2XS

Schedule

Save*

Comments

Share

1 | SELECT * FROM atliq.streaming.gold_daily_summary ORDER BY event_date DESC;

Add parameter

Table

	event_date	orders_placed	orders_paid	orders_cancelled	.00 revenue	refreshed_at
1	2026-10-01	1748	1445	323	3778163.00	2026-10-01T22:21:39.919+00:...

1 row | 0.672s runtime | See performance (1) | Checked for improvements

Refreshed at 10/2/26, 3:55 AM (Now)

gold_daily_summary rebuilt by the DAG: refreshed_at 22:21:39 UTC.



STEP 6

A Real Retry, Unstaged

WHAT

Unpausing the DAG also started the latest hourly slot, which overlapped the manual run. Both tried to rebuild `gold_daily_summary` at once; Delta rejected one write, and Airflow retried it five minutes later with success.

WHY

This was not planned. Delta refused a conflicting write rather than corrupt the table, and the retry policy in `default_args` recovered with no human involved. That is what retries are for in production.

STEP 6 | A Real Retry, Unstaged

refresh_daily_summary - Airflow

Query History - Databricks

localhost:8080/dags/atliq_streaming_ops/runs/scheduled_2026-10-01T22:00:00+00:00/tasks/refresh_daily_summary

Ask Gemini

atliq_streaming_ops

✓ scheduled_2026-10-01T22:00:00+00:00

Task refresh_daily_summary

Home

Dags

Assets

Browse

Admin

Security

Docs

+05:30

User

check_fresh_events

DatabricksSqlOperator

✓ success

optimize_tables

DatabricksSqlOperator

✓ success

refresh_daily_summary

DatabricksSqlOperator

✓ success

refresh_daily_summary

✓ Success

✓ / ✕

Operator

Try Number

Start Date

DatabricksSqlOperator

2

2026-10-02 03:56:41

End Date

Duration

Dag Version

2026-10-02 03:56:47

00:00:06.383

v1

Add a note...

Task Tries

1 ✕

2 ✓

All Log Levels

All Sources

Search logs...

GROUP BY to_date(event_ts), parameters: None

6 [2026-10-02 03:56:41] WARNING - ProvidersManager.hooks is depr

7 [2026-10-02 03:56:43] INFO - Received status code 200 for POST

8 [2026-10-02 03:56:43] INFO - HTTP Response with status code 200

9 [2026-10-02 03:56:43] INFO - Successfully opened session 01f1b

10 [2026-10-02 03:56:44] INFO - Received status code 200 for GET

11 [2026-10-02 03:56:44] INFO - Running statement: CREATE OR REPL

SELECT

to_date(event_ts)

COUNT_IF(event_type = 'order_placed')

COUNT_IF(event_type = 'payment_received')

COUNT_IF(event_type = 'order_cancelled')

SUM(CASE WHEN event_type = 'payment_received'

THEN order_amount ELSE 0 END)

current_timestamp()

FROM atliq.streaming.silver_order_events

GROUP BY to_date(event_ts), parameters: None

12 [2026-10-02 03:56:46] INFO - Received status code 200 for POST

13 [2026-10-02 03:56:46] INFO - HTTP Response with status code 200

14 [2026-10-02 03:56:47] INFO - Databricks query id: 01f1bde7-2fd

15 [2026-10-02 03:56:47] INFO - Closing session 01f1bde7-2f11-11e

16 [2026-10-02 03:56:47] INFO - Received status code 200 for

17 [2026-10-02 03:56:47] INFO - HTTP Response with status cod

Post Execute

refresh_daily_summary in the scheduled run: try 1 failed, try 2 succeeded at 03:56:41.



STEP 7

Maintenance on Serverless

WHAT

Silver's Delta history shows OPTIMIZE entries seconds after each streaming write, compacting 128 or 244 small files into one, before the DAG ever ran.

WHY

Databricks auto-compacts managed tables, so the DAG's OPTIMIZE on Silver was a safe no-op, while on Gold it still compacted 5 files into 1. The scheduled task is idempotent: near free when there is nothing to do, insurance where auto-compaction is off.

STEP 7 | Maintenance on Serverless

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Agents

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Visual Data Prep

AI/ML

Playground

Agents

AI Gateway

Experiments

Features

Models

Serving

SQL Editor

Tree view: ON

Workspace

Type to search

Repos

Shared

Users

daily_summary

optimize_history

Run all (1000)

Just now (24)

workspace.default

Last edit was now

Generate

Serverless Starte...

2XS

Schedule

Save*

Comments

Share

1

2

3

4

5

SELECT version, timestamp, operation,

operationMetrics.numRemovedFiles,

operationMetrics.numAddedFiles

FROM (DESCRIBE HISTORY atliq.streaming.silver_order_events)

ORDER BY version DESC LIMIT 8;

Add parameter

Table

version

timestamp

operation

numRemovedFiles

numAddedFiles

1

13

2026-10-01T20:57:51.000+00:...

OPTIMIZE

244

1

2

12

2026-10-01T20:44:13.000+00:...

STREAMING UPDATE

0

0

3

11

2026-10-01T20:44:12.000+00:...

OPTIMIZE

128

1

4

10

2026-10-01T20:44:05.000+00:...

STREAMING UPDATE

0

200

5

9

2026-10-01T20:25:18.000+00:...

OPTIMIZE

128

1

6

8

2026-10-01T20:25:15.000+00:...

STREAMING UPDATE

0

0

7

7

2026-10-01T20:25:04.000+00:...

STREAMING UPDATE

0

193

8

6

2026-10-01T19:24:37.000+00:...

STREAMING UPDATE

0

0

8 rows

2.423s runtime

See performance (1)

Checked for improvements

Refreshed at 10/2/26, 3:59 AM (Now)

silver_order_events history: automatic OPTIMIZE after each STREAMING UPDATE.



STEP 8

The Gate Fails on Stale Data

WHAT

With no new events, Silver aged past the limit: `minutes_old` reached 124. Triggered the DAG again; `check_fresh_events` failed and both downstream tasks were marked `upstream_failed`.

WHY

A quality gate that cannot fail is not a gate. The run failed in 7.7 seconds with no retry delay, and neither `OPTIMIZE` nor the rollup ran on stale data.

STEP 8 | The Gate Fails on Stale Data (1 of 2)

databricks

Verify identity workspace

+ New

- Home
- Learn
- Workspace
- Recents
- Catalog
- Jobs & Pipelines
- Compute
- Discover
- Marketplace

SQL

SQL Editor

- Queries
- Dashboards
- Genie Agents
- Alerts
- Query History
- SQL Warehouses

Data Engineering

- Runs
- Data Ingestion
- Visual Data Prep

AI/ML

- Playground
- Agents
- AI Gateway
- Experiments
- Features
- Models
- Serving

Tree view: ON

Type to search

- Drafts
 - 01_kafka_stream_processing
 - Bakehouse Sales Starter Space
 - daily_summary
 - Min_old**
 - optimize_history

daily_summary optimize_history Min_old +

Run all (1000) Just now (15s) workspace default Last edit was now

```

1 SELECT MAX(event_ts) AS last_event,
2       current_timestamp() AS now,
3       timestampdiff(MINUTE, MAX(event_ts), current_timestamp()) AS minutes_old
4 FROM atliq.streaming.silver_order_events;
```

Add parameter

Table	last_event	now	minutes_old
1	2026-10-01T20:39:34.502+00:00	2026-10-01T22:43:55.149+00:00	124

1 row | 15.323s runtime See performance (1) Checked for improvements

Generate Serverless Starte... 2XS Schedule Save* Comments Share

Refreshed at 10/2/26, 4:13 AM (Now)

Before the red run: minutes_old 124, past the 120-minute limit.

STEP 8 | The Gate Fails on Stale Data (2 of 2)

manual__2026-10-01T22:46:25...

Min_old - Databricks

localhost:8080/dags/atliq_streaming_ops/runs/manual__2026-10-01T22:46:25.817864+00:00?run_after_lte=2026-10-01T22%3A46%3A25.817864Z

Ask Gemini

atliq_streaming_ops

Dag Run
manual__2026-10-01T22:46:25.817864+00:00

Home

Dags

Assets

Browse

Admin

Security

Docs

+05:30

User

check_fresh_events

DatabricksSqlOperator

failed

optimize_tables

DatabricksSqlOperator

upstream_failed

refresh_daily_summary

DatabricksSqlOperator

upstream_failed

Search Dags Ctrl+K

Trigger

manual__2026-10-01T22:46:25.817864+00:00

Failed

Logical Date

Run Type

Start Date

2026-10-02 04:16:16

Manual

2026-10-02 04:16:26

End Date

Duration

Triggering User

Dag Version(s)

2026-10-02 04:16:33

00:00:07.743

airflow

v1

Add a note...

+ Filter

3 Task Instances

Task ID

Map Index

State

refresh_daily_summary

Upstream F

optimize_tables

Upstream F

check_fresh_events

Failed

React Flow

check_fresh_events failed; optimize_tables and refresh_daily_summary upstream_failed; run failed in 7.7 s.



STEP 9

Why It Failed, in the Log

WHAT

The task log shows Airflow sending the freshness query and Databricks answering with `USER_RAISED_EXCEPTION: DQ FAIL`, no events in `silver_order_events` in the last 2 hours.

WHY

assert_true makes the warehouse raise the error, and Airflow turns it into a failed task with a readable reason. Whoever is on call sees exactly what is wrong without opening Databricks.

STEP 9 | Why It Failed, in the Log

The screenshot displays the Apache Airflow web interface. On the left, a sidebar contains navigation links for Home, Dags, Assets, Browse, Admin, and Security. The main area shows a DAG named 'atliq_streaming_ops' with a failed run 'manual_2026-10-01T22:46:25.817864+00:00'. The task 'check_fresh_events' is highlighted, showing it failed with an 'upstream_failed' status. The logs for this task are displayed on the right, showing a series of INFO messages followed by an ERROR message: 'Task failed with exception'. The error message is a 'DatabricksSqlExecutionError' with the message 'Error running SQL statement: SELECT assert_true(COUNT(*) > 0, 'DQ FAIL: no events in atliq.streaming.silver_order_events in the last 2 hours') AS freshness_ok FROM atliq.streaming.silver_order_events WHERE event_ts >= current_timestamp() - INTERVAL 2 HOURS, parameters: None'. The logs also show the file paths for the Airflow SDK and Databricks providers.

check_fresh_events log: ServerOperationError [USER_RAISED_EXCEPTION] DQ FAIL.

Task 4 Complete

What this task delivered

Orchestration that runs itself

Airflow 3 in Docker runs an hourly DAG against Databricks with an encrypted, SQL-scoped connection.

A gate proven both ways

Green at 99 minutes old, red at 124: the freshness check passes and fails exactly as designed, and blocks downstream work.

Resilience on record

A real write conflict recovered by retry; auto-compaction spotted and explained; every statement traced in Query History.

Run evidence

24.8 s

green run

7.7 s

red run, fail fast

2 tries

retry recovered

7

Airflow services

Engineering decisions: Gate first in the chain so nothing runs on stale data; retries=0 on the gate, two retries elsewhere; token scoped to SQL and stored encrypted with a Fernet key set before first start; DatabricksSqlOperator on the serverless warehouse; daily rollup rebuilt in full each run so it is idempotent; DAG paused after capture.